Name: Ramavath Prashanth Naik;

Reg.no:-192472272;

Robot Navigation Using Reinforcement Learning: TD(0), SARSA, and Q-Learning

Proposed Algorithm for Robot Navigation Using Reinforcement Learning

Step 1: Define Robot Environment and State Space

The robot navigation environment is represented as a grid-based world where the robot learns to reach the target destination while avoiding obstacles.

Let the environment be defined as:

$$E = (S, A, R, P, \gamma)$$

(1)

Where:

- S= Set of possible robot states (positions)
- A= Available actions (up, down, left, right)
- R= Reward obtained after taking an action

- P= Transition probability between states
- γ= Discount factor

Eq. (1) defines the Markov Decision Process (MDP) representation of the robot navigation problem. It describes how the robot interacts with its environment and learns optimal movements.

Step 2: Initialize Robot Parameters

For every robot state and action pair, initialize the Q-value table.

$$Q(s, a) = 0$$

(2)

Where:

- s= Current robot state
- a= Selected action
- $Q(s, a)$= Expected future reward

Eq. (2) initializes the learning table before navigation training begins. Initially, the robot has no knowledge about the environment.

Step 3: TD(0) Learning Algorithm

TD(0) updates the value function by using the difference between predicted and received rewards.

The Temporal Difference error is calculated as:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

(3)

Where:

- R_{t+1}= Immediate reward
- $V(S_t)$= Current state value
- $V(S_{t+1})$= Next state value

Eq. (3) calculates the prediction error between current estimation and future reward. TD(0) improves robot decision-making by continuously correcting state values.

The value update equation is:

$$V(S_t) = V(S_t) + \alpha\delta_t$$

(4)

Where:

- α= Learning rate

Eq. (4) updates the robot's state value based on the observed navigation experience.

Step 4: SARSA Algorithm (On-Policy Learning)

SARSA updates Q-values based on the action actually selected by the robot.

The SARSA update equation is:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

(5)

Where:

- S_t= Current state
- A_t= Current action
- A_{t+1}= Next selected action

Eq. (5) updates the robot path according to the current navigation policy. SARSA learns safer paths because it considers the actual action taken by the robot.

Step 5: Q-Learning Algorithm (Off-Policy Learning)

Q-learning selects the maximum future reward action independent of the current policy.

The Q-learning update equation is:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

(6)

Where:

- $max_a Q(S_{t+1}, a)$represents the best possible future action.

Eq. (6) updates the robot's Q-table by selecting the action with maximum expected reward. It helps the robot discover the shortest optimal path.

Step 6: Exploration and Exploitation Strategy

The robot balances between exploring new paths and selecting learned optimal paths.

Using ε-greedy strategy:

$$A = \begin{cases} Random\ Action, & Probability \epsilon \\ Best\ Action, & Probability(1-\epsilon) \end{cases}$$

(7)

Eq. (7) allows the robot to explore unknown areas initially and gradually exploit the learned optimal navigation path.

Step 7: Comparison of TD(0), SARSA, and Q-Learning

Algorithm	Learning Type	Update Equation	Policy	Navigation Behaviour
TD(0)	Value-based learning	Eq. (4)	State Value	Learns value of each position
SARSA	On-policy learning	Eq. (5)	Current policy	Safer path selection
Q-Learning	Off-policy learning	Eq. (6)	Optimal policy	Fast shortest-path learning

Step 8: Numerical Calculation Example

Consider a robot at state:

$$S_t = (2, 2)$$

The robot moves right.

Given:

- Reward $R = 10$
- Learning rate $\alpha = 0.5$
- Discount factor $\gamma = 0.9$
- Current Q value:

$$Q(S_t, A_t) = 4$$

Future maximum Q value:

$$maxQ(S_{t+1}, a) = 8$$

Using Q-learning equation:

$$Q_{new} = 4 + 0.5[10 + 0.9(8) - 4]$$

(8)

Calculation:

$$Q_{new} = 4 + 0.5[10 + 7.2 - 4]$$
$$Q_{new} = 4 + 0.5(13.2)$$
$$Q_{new} = 4 + 6.6$$
$$Q_{new} = 10.6$$

Eq. (8) shows that the robot increases the Q-value of the selected path because it produced a high reward. The

robot will prefer this path in future navigation episodes.

Step 9: Prediction and Final Navigation Decision

After multiple training episodes:

$$Optimal\ Path = argmax_a Q(S, a)$$

(9)

Where:

- $argmax$selects the action with the highest learned Q-value.

Eq. (9) predicts the final robot movement by selecting the action with maximum expected reward from the trained Q-table.

Final Prediction Process

1. Robot starts from the initial position.
2. It explores different paths.
3. TD(0) improves state value estimation.

4. SARSA learns safe navigation based on actual movements.
5. Q-learning finds the shortest optimal route.
6. After training, the robot selects:

$$Highest\ Q - value \rightarrow Best\ Action$$

7. Robot reaches the destination with minimum distance and minimum collision risk.